Вы не знаете JS: Область видимости и замыкания

Глава 1: Что такое область видимости?

Одна из самых фундаментальных парадигм почти всех языков программирования — возможность сохранять значения в переменных, а позже извлекать или менять эти значения. Собственно, возможность хранить значения и извлекать значения из переменных — это то, что дает программе *состояние*.

Без такого концепта программа могла бы выполнять некоторые задачи, но они были бы весьма ограничены и не были бы очень уж интересны.

Но включение переменных в нашу программу порождает самые интересные вопросы, которые мы теперь зададим: где эти переменные *живут*? Другими словами, где они хранятся? И, что более важно, как наша программа их находит, когда нуждается в них?

Эти вопросы говорят о необходимости четкого набора правил для хранения переменных в некотором месте и для обнаружения этих переменных позднее. Мы назовем этот набор правил — *Область видимости*.

Но где и как правила этих *областей видимости* устанавливаются?

Теория компиляторов

Само собой разумеется, хотя это может удивить в зависимости от вашего уровня взаимодействия с различными языками, но несмотря на тот факт, что JavaScript попадает под общую категорию "динамических" или "интерпретируемых" языков, на самом деле он язык компилируемый. Он *не* компилируется заранее, как многие традиционно компилируемые языки, и результаты компиляции не являются переносимыми среди различных распределенных систем.

Тем не менее, среда исполнения JavaScript выполняет много тех шагов, что и любой другой традиционный компилятор языка. Хоть и более сложными

способами, чем мы обычно можем себе представить.

В традиционном процессе языковой компиляции, часть кода вашей программы обычно проходит три шага *до* того, как будет выполнена, в общих чертах называемых "компиляцией":

1. **Разбиение на лексемы (Tokenizing/Lexing)**: разбиение строки символов на имеющие смысл (для языка) части, называемые лексемами. Например, представьте программу: `var a = 2;`. Эта программа вероятнее всего будет разбита на следующие лексемы: `var`, `a`, `=`, `2` и `;`. Пробел может быть сохранен или не сохранен как лексема, в зависимости от того, имеет он смысл или нет.

   Примечание: Разница между *tokenizing* и *lexing* — едва различима и теоретическая, но она сосредотачивается на том, идентифицируются ли эти лексемы как *без состояния* или *с состоянием*. Проще говоря, если токенизатор используется для того, чтобы вызывать правила парсинга с сохранением состояния чтобы выяснить, следует ли считать `a` отдельной лексемой или только частью другой лексемы, то *это* будет **lexing**.

2. **Парсинг:** берет поток (массив) лексем и превращает его в дерево вложенных элементов, которые сообща представляют грамматическую структуру программы. Это дерево называется "AST" (**A**bstract **S**yntax **T**ree, дерево абстрактного синтаксиса).

   Такое дерево для `var a = 2;` может начинаться с узла верхнего уровня с названием `VariableDeclaration`, с дочерним узлом `Identifier` (чье значение равно `a`) и еще одним дочерним узлом `AssignmentExpression`, у которого тоже есть дочерний узел `NumericLiteral` (чье значение равно `2`).

3. **Генерация кода:** процесс взятия AST и превращения его в исполняемый код. Эта часть сильно зависит от языка, платформы назначения и т.п..

   Итак, вместо того, чтобы увязать в деталях, мы просто опустим их и скажем, что есть способ взять наше вышеописанное AST для `var a = 2;` и превратить его в набор машинных инструкций, чтобы в действительности *создать* переменную с именем `a` (включая выделение памяти и т.д.), а затем сохранить значение в `a`.

   Примечание: подробности того, как движок управляет системными ресурсами, глубже, чем мы будем "копать". Поэтому мы всего лишь

примем на веру, что движок умеет создавать и сохранять переменные когда это необходимо.

Движок JavaScript гораздо сложнее, чем *просто* эти три шага, как и большинство других языковых компиляторов. Например, в процессе синтаксического анализа и генерации кода безусловно существуют шаги по оптимизации быстродействия выполнения, включая сокращение избыточных элементов и т.п..

Так что здесь я лишь очерчиваю границы. Но я думаю, что вскоре вы поймете, почему эти детали, которые мы *сейчас* освещаем, даже на высоком уровне, актуальны.

Прежде всего, JavaScript-движки не могут позволить себе роскошь (как и другие языковые компиляторы) иметь массу времени на оптимизацию. Потому что компиляция JavaScript не происходит заблаговременно на этапе сборки, как это происходит в других языках.

Для JavaScript, компиляция во многих случаях происходит всего лишь за микросекунды (или меньше!) перед выполнением кода. Чтобы гарантировать высочайшее быстродействие, движки JS используют все виды уловок (такие как JIT, который компилирует лениво и даже перекомпилирует на ходу), которые вне "области" нашего обсуждения тут.

Давайте скажем, простоты ради, что любой фрагмент JavaScript-кода должен быть скомпилирован до (обычно *прямо* перед этим!) его выполнения. Таким образом, компилятор JS *сперва* возьмет программу `var a = 2;` и скомпилирует ее, а затем будет готов выполнить ее. Обычно сразу же.

Понимание области видимости

Мы будем подходить к изучению темы области видимости таким образом, чтобы думать об этом процессе в терминах диалога. Но *кто* же ведет этот диалог?

Действующие лица

Давайте познакомимся с действующими лицами, которые взаимодействуют при обработке программы `var a = 2;`, для того чтобы понимать их разговоры, которые мы вскоре услышим:

1. *Движок* отвечает за компиляцию от начала и до конца, а также за выполнение нашей JavaScript программы;
2. *Компилятор* — один из друзей *Движка*, выполняет всю грязную работу по синтаксическому анализу и генерации кода (см. предыдущий раздел);
3. *Область видимости* — еще один друг *Движка*, собирает и обслуживает список поиска всех объявленных идентификаторов (переменных) и следит за исполнением строгого набора правил относительно того, каким образом эти идентификаторы доступны для текущего выполняемого кода.

Для *полного понимания* как работает JavaScript, вам необходимо начать *думать* как *Движок* (и его друзья) думают, задавать вопросы как они задают и отвечать на них точно так же.

Туда и обратно

Когда вы видите программу `var a = 2;`, вы вероятнее всего подумаете о ней как об одном операторе. Но наш новый друг *Движок* видит это не так. На самом деле, *Движок* видит два отдельных оператора: один, который *Компилятор* обработает во время компиляции, а другой, который *Движок* обработает во время выполнения.

Так давайте же разберем по полочкам, как *Движок* и его друзья поступят с программой `var a = 2;`.

Первое, что сделает *Компилятор* с этой программой, — это разобъет ее на лексемы, которые затем распарсит в дерево. Но когда *Компилятор* доберется до генерации кода, он будет интерпретировать программу несколько по-иному нежели предполагалось.

Разумным предположением могло бы быть то, что *Компилятор* будет производить код, который можно кратко представить следующим псевдокодом: "Выделить память для переменной, пометить ее как `a`, затем поместить значение `2` в эту переменную." К сожалению, это не совсем точно.

Компилятор вместо этого сделает следующее:

1. Встретив `var a`, *Компилятор* попросит *Область видимости* проверить, существует ли уже переменная `a` в коллекции указанной области

видимости. Если да, то *Компилятор* проигнорирует это объявление переменной и двинется дальше. В противном случае, *Компилятор* попросит *Область видимости* объявить новую переменную `a` в коллекции указанной области видимости.

2. Затем *Компилятор* сгенерирует код для *Движка* для последующего выполнения, чтобы обработать присваивание `a = 2`. Код, который *Движок* запускает, сначала спросит *Область видимости* есть ли переменная с именем `a`, доступная в коллекции текущей области видимости. Если есть, то *Движок* будет использовать эту переменную. Если нет, то *Движок* будет искать *в другом месте* (см. раздел *Вложенная область видимости* ниже).

Если *Движок* в итоге найдет переменную, он присвоит ей значение `2`. Если нет, то *Движок* вскинет руки и выкрикнет: "Ошибка!".

Подводя итог, чтобы присвоить значение переменной, выполняются два различных действия: во-первых, *Компилятор* объявляет переменную (если она не была объявлена ранее) в текущей области видимости, а во-вторых, при выполнении кода, *Движок* ищет эту переменную в *Области видимости* и, если находит, присваивает ей значение.

Компилятор расскажет

Нам нужно еще немного компиляторной терминологии, перед тем как двинуться дальше.

Когда *Движок* выполняет код, который *Компилятор* генерирует на шаге (2), он должен найти переменную `a`, чтобы увидеть, была ли она объявлена. И этот поиск принимает во внимание *Область видимости*. Но тип поиска, который выполняет *Движок*, влияет на результат поиска.

В нашем случае говорят, что *Движок* будет выполнять "LHS"-поиск переменной `a`. Другой тип поиска называется "RHS".

Держу пари, что вы можете угадать что означают "L" и "R". Эти термины означают "Left-hand Side" (левая сторона) и "Right-hand Side" (правая сторона).

Сторона... чего? *Операции присваивания.*

Иными словами, LHS-поиск выполняется, когда переменная появляется с левой стороны операции присваивания, а RHS-поиск выполняется, когда переменная появляется с правой стороны операции присваивания.

На самом деле, давайте будем немного точнее. RHS-поиск, для наших целей, неотличим от простого поиска значения некоторой переменной. Тогда как LHS-поиск пытается найти сам контейнер переменной, чтобы он мог присвоить значение. Таким образом, RHS по существу *не обязательно* означает "правая сторона присваивания", он просто более точно означает "не левая сторона".

Став на мгновение легкомысленным, вы могли бы подумать, что RHS вместо этого означает "извлеки его/ее исходное значение", подразумевая, что RHS — это "иди и получи значение из...".

Давайте копнем немного глубже в этом направлении.

Когда я говорю:

```
console.log( a );
```

Ссылка на `a` — это RHS-ссылка, потому что здесь ничего не присваивается в `a`. Напротив, мы выполняем поиск, чтобы извлечь значение `a`, для того, чтобы передать значение в `console.log(..)`.

Для сравнения:

```
a = 2;
```

Ссылка на `a` здесь — это LHS-ссылка, так как мы не заботимся здесь о том, каково текущее значение, мы просто хотим найти эту переменную как цель для операции присваивания `= 2`.

Примечание: LHS и RHS, означающие "левая/правая сторона присваивания", не обязательно буквально означают "левая/правая сторона операции присваивания `=`". Есть еще несколько способов, которыми производится присваивание, и поэтому лучше концептуально думать о нем как: "кто является целью присваивания (LHS)" и "кто источник присваивания (RHS)".

Представьте такую программу, в которой есть обе ссылки LHS и RHS:

```
function foo(a) {
    console.log( a ); // 2
}

foo( 2 );
```

Последняя строка, которая активизирует `foo(..)` как вызов функции, требует RHS-ссылку на `foo`, что значит, "сходи и найди значение `foo` и дай его мне". Более того, `(..)` означает, что значение `foo` должно быть выполнено, поэтому это скорее всего функция!

Здесь есть едва уловимое, но важное присваивание. **Вы обнаружили его?**

Вы наверное упустили неявное `a = 2` в этом коде. Это происходит, когда значение `2` передается как аргумент в функцию `foo(..)`, в этом случае значение `2` **присваивается** параметру `a`. Чтобы (неявно) присвоить значение параметру `a`, выполняется LHS-поиск.

Также есть и RHS-ссылка на значение `a` и это результирующее значение передается в `console.log(..)`. `console.log(..)` нужна ссылка для выполнения. Для объекта `console` это RHS-поиск, затем происходит разрешение имени свойства чтобы убедиться существует ли метод, называемый `log`.

Наконец, мы можем осмыслить, что есть LHS/RHS-обмен передаваемым значением `2` (путем RHS-поиска переменной `a`) в `log(..)`. Внутри родной реализации `log(..)`, мы можем предположить, что у нее есть параметры, у первого из которых (возможно называющегося `arg1`) есть поиск LHS-ссылки, до присваивания ему `2`.

Примечание: У вас может появиться соблазн представлять объявление функции `function foo(a) {...` как обычное объявление переменной и присваивание, такое как `var foo` и `foo = function(a){...`. Делая так, будет соблазн думать об объявлении этой функции как подразумевающей LHS-поиск.

Однако, едва заметная, но важная разница есть в том, что *Компилятор* обрабатывает как объявление, так и определение значения во время генерации кода, благодаря чему, когда *Движок* выполняет код, не требуется никакой обработки чтобы "присвоить" значение функции в `foo`. Следовательно, неуместно думать об объявлении функции как о

присваивании с помощью LHS-поиска тем способом, который мы здесь обсуждаем.

Беседа Движка и Области видимости

```
function foo(a) {
    console.log( a ); // 2
}

foo( 2 );
```

Давайте представим вышеуказанный обмен между ними (который обрабатывает этот код) как беседу. Беседа может пойти примерно так:

> Движок: Эй, Область видимости, у меня есть RHS-ссылка на foo. Когда-нибудь слышала о такой?

> Область видимости: Ну разумеется,
> слышала. Компилятор объявил ее всего секунду назад. Это функция. Пожалуйста!

> Движок: Отлично, спасибо! Хорошо, я выполняю foo.

> Движок: Эй, Область видимости, у меня есть LHS-ссылка на a, слышала что-нибудь о ней?

> Область видимости: Ну разумеется,
> слышала. Компилятор объявил ее как формальный параметр в foo только что. Пожалуйста!

> Движок: Отзывчива как всегда, Область видимости. Снова спасибо. А теперь присвоим 2 в a.

> Движок: Эй, Область видимости, извини, что беспокою тебя снова. Мне нужен RHS-поиск console. Когда-нибудь слышала о таком имени?

> Область видимости: Нет проблем, Движок, это то, чем я весь день и занимаюсь. Да, у меня есть console. Она встроенная. Пожалуйста!

> Движок: Идеально. Ищу log(..). Превосходно, это функция.

> Движок: Эй, Область видимости. Можешь помочь мне с RHS-ссылкой на a? Думаю, я ее помню, но просто хочу лишний раз проверить.

> Область видимости: Ты прав, Движок. Та же ссылка, не изменилась. Пожалуйста!

> Движок: Круто! Передаю значение a, которое равно 2, в log(..).

> ...

Тест

Проверьте ваше понимание на настоящий момент. Обязательно сыграйте роль *Движка* и поучаствуйте в "беседе" с *Областью видимости*:

```
function foo(a) {
    var b = a;
    return a + b;
}

var c = foo( 2 );
```

1. Определите все LHS-поиски (их 3!).
2. Определите все RHS-поиски (их 4!).

Примечание: См. обзор этой главы, чтобы узнать ответы на тест!

Вложенная область видимости

Мы говорили, что *Область видимости* — это набор правил поиска переменных по их идентификатору. Однако, обычно бывает более одной *Области видимости*.

Также как блок или функция вкладывается внутрь другого блока или функции, области видимости вкладываются внутрь других областей. Поэтому, если переменную не найти в ближайшей области видимости, *Движок* заглядывает в следующую внешнюю по отношению к этой область видимости, продолжая так до тех пор, пока не найдет или пока не достигнет самой внешней (т.е. глобальной) области.

Пример:

```
function foo(a) {
    console.log( a + b );
}

var b = 2;

foo( 2 ); // 4
```

RHS-ссылка на `b` не может быть разрешена внутри функции `foo`, но она может быть разрешена в *Области видимости*, окружающей ее (в этом случае, глобальной).

Поэтому, еще раз пересмотрев беседы между *Движком* и *Областью видимости*, мы возможно услышим:

> Движок: "Эй, Область видимости foo, что-нибудь слышала о b? У меня есть RHS-ссылка на нее".

> Область видимости: "Не-а, никогда не слышала о такой. Попробуй что-нибудь другое!"

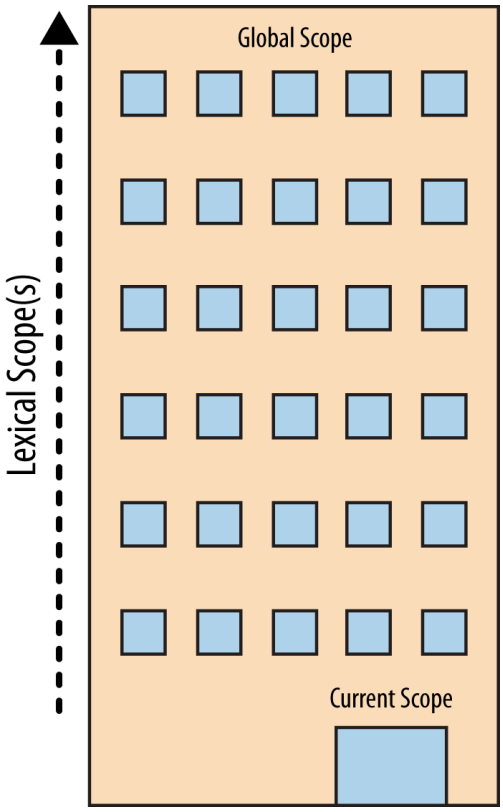
> Движок: "Эй, Область видимости снаружи foo! О, ты еще и глобальная Область видимости, круто. Когда-нибудь слышала о b? У меня есть RHS-ссылка на нее."

> Область видимости: "Да-да, конечно есть. Пожалуйста!"

Простые правила просмотра вложенных *Областей видимости*: *Движок* начинает в текущей выполняемой *Области видимости*, ищет в ней переменную, затем если не находит, продолжает поиск уровнем выше и так далее. Если достигнута самая внешняя глобальная область видимости, поиск останавливается, независимо от того, нашел он переменную или нет.

Берем за основу метафоры

Для визуализации процесса разрешения во вложенных *Областях видимости*, я хочу, чтобы вы подумали об этом высоком здании.



Здание символизирует набор правил вложенных *Областей видимости* нашей программы. Первый этаж здания представляет вашу текущую выполняемую *Область видимости*, где бы вы ни были. Верхний уровень здания — это глобальная *Область видимости*.

Вы разрешаете LHS- и RHS-ссылки ища на вашем текущем этаже, а если вы не нашли что искали, поднимаетесь на лифте на следующий этаж, ища там, затем на следующий и так далее. Как только вы попадаете на верхний этаж(глобальная *Область видимости*), вы либо находите то, что искали, либо не находите. Но в любом случае вы должны остановиться.

Ошибки

Почему имеет значение называть поиск LHS или RHS?

Потому что эти два типа поиска ведут себя по-разному в обстановке, когда переменная еще не была объявлена (не была найдена ни в одной просмотренной *Области видимости*).

Представьте:

```
function foo(a) {
    console.log( a + b );
    b = a;
}

foo( 2 );
```

Когда происходит RHS-поиск `b` первый раз, она не будет найдена. Это как бы "необъявленная" переменная, так как она не была найдена в этой области видимости.

Если RHS-поиск не сможет когда-либо найти переменную, в любой из вложенных *Областей видимости*, это приведет к возврату *Движком* ошибки `ReferenceError`. Важно отметить, что эта ошибка имеет тип `ReferenceError`.

Напротив, если *Движок* выполняет LHS-поиск и достигает верхнего этажа (глобальной *Области видимости*) и не находит ничего, и если программа не запущена в "строгом режиме", то затем глобальная *Область видимости* создаст новую переменную с таким именем **в глобальной области видимости** и передаст ее обратно *Движку*.

"Нет, до этого не было ни одной и я любезно создала ее для тебя."

"Строгий режим", который был добавлен в ES5, имеет ряд разных отличий от обычного/нестрогого/ленивого режима. Одно такое отличие — это то, что он запрещает автоматическое/неявное создание глобальных переменных. В этом случае, не было бы никакой переменной в глобальной *Области видимости*, чтобы передать обратно от LHS-поиска, и *Движок* выбросит `ReferenceError` аналогично случаю с RHS.

Теперь, если переменная найдена в ходе RHS-поиска, но вы пытаетесь сделать что-то с ее значением, что невозможно, например, пытаетесь выполнить как функцию не-функциональное значение или ссылаетесь на свойство значения `null` или `undefined`, то *Движок* выдаст другой вид ошибки, называемый `TypeError`.

`ReferenceError` — это сбой разрешения имени, связанный с *Областью видимости*, тогда как `TypeError` подразумевает, что разрешение имени в *Области*

видимости было успешным, но была попытка выполнения нелегального/невозможного действия с результатом.

Обзор

Область видимости — это набор правил, которые определяют где и как переменная (идентификатор) могут быть найдены. Этот поиск может осуществляться для целей присваивания значения переменной, которая является LHS (left-hand-side) ссылкой, или может осуществляться для целей извлечения ее значения, которое является RHS (right-hand-side) ссылкой.

LHS-ссылки являются результатом операции присваивания. Присваивания, связанные с *Областью видимости*, могут происходить либо с помощью операции `=`, либо передачей аргументов (присваиванием) параметрам функции.

JavaScript *Движок* перед выполнением сначала компилирует код, и пока он это делает, он разбивает операторы, подобные `var a = 2;` на два отдельных шага:

1. Первый, `var a`, чтобы объявить ее в *Область видимости*. Это выполняется в самом начале, до исполнения кода.
2. Позже, `a = 2` ищет переменную (LHS-ссылку) и присваивает ей значение, если находит.

Оба поиска ссылок LHS и RHS начинаются в текущей выполняющейся *Области видимости* и если нужно (т.е. они не нашли что искали в ней), они работают с их более высокими вложенными *Областями видимости*, с одной областью (этажом) за раз, ища идентификатор, пока не доберутся до глобальной (верхний этаж) и не остановятся, вне зависимости от результата поиска.

Невыполненные RHS-ссылки приводят к выбросу `ReferenceError`. Невыполненные LHS-ссылки приводят к автоматической, неявно созданной переменной с таким именем (если не включен "Строгий режим"), либо к `ReferenceError` (если включен "Строгий режим").

Ответы к тесту

```
function foo(a) {
    var b = a;
    return a + b;
}

var c = foo( 2 );
```

1. Определите все LHS-поиски (их 3!).

   `c = ..` **,** `a = 2` **(неявное присваивание параметру) и** `b = ..`

2. Определите все RHS-поиски (их 4!).

   `foo(2..` **,** `= a;` **,** `a + ..` **и** `.. + b`

Про "Строгий режим" см. здесь